UNIVERSAL CREATIVE ENGINE (UCE)

V0 Technical Design Specification

Version 0.1 | 2026-09-15 | Confidential Working Baseline

Document Purpose

This document translates the UCE Software Requirements Specification into concrete engineering contracts for the first buildable architecture. It is normative for V0 unless superseded by an approved Architecture Decision Record (ADR).

Field	Value
Target release	UCE V0 architecture-proving prototype
Primary workflow	Import -> Select -> UEI -> Track -> Remove -> Verify -> Repair -> Export
Primary deployment	Single-user desktop, local-first
Reference platforms	Windows 11 + NVIDIA GPU first; Linux development supported
Excluded from V0	Replacement, insertion, full 3D editing, multi-user collaboration, cloud rendering, arbitrary agent autonomy

1. V0 Engineering Goals

- Prove that UCE project semantics remain independent from any one AI model.
- Prove Universal Entity Identity can survive segmentation/tracking operations without being owned by a tracker.
- Prove non-destructive operations, CAS-backed artifacts, transactional saves, and rollback/recovery.
- Prove local-only privacy: the complete V0 workflow must succeed with networking blocked.
- Prove worker isolation: an AI worker failure must not corrupt project state or terminate the desktop shell.
- Prove measurable edit quality through preservation, temporal consistency, and defect-localization checks.

2. V0 System Boundary

```
UCE Desktop (trusted)
  |-- UI / Project Controller
  |-- Policy Engine
  |-- Operation Coordinator
  |-- Local Project Store
  |
  +--> Media Worker (isolated)
  +--> Vision Worker (isolated)
  +--> Inpaint Worker (isolated)
  +--> Quality Worker (isolated)
  +--> Export Worker (isolated)
```

Authoritative state: project DB + USD/semantic state + operation log
Large immutable data: CAS
Workers produce artifacts; only Project Controller commits authoritative state.

3. Process Architecture

Process	Trust	Responsibilities	Must not do
Desktop/UI	Trusted	Project lifecycle, UI, policy decisions,	Run arbitrary third-party

		operation orchestration	model code in-process
Media Worker	Restricted	Probe/decode/encode/ proxy/audio muxing	Mutate project metadata directly
Vision Worker	Restricted	Segmentation, temporal propagation, tracking evidence	Own UEI identity
Inpaint Worker	Restricted	Reconstruction/ inpainting candidate generation	Commit final state
Quality Worker	Restricted	Preservation/temporal/ defect scoring	Overwrite source or operation history
Export Worker	Restricted	Final encode/mux/provenance packaging	Read unrelated projects

V0 workers MAY initially be Python subprocesses using a versioned JSON-RPC or gRPC-like local contract. The production transport is intentionally not frozen until profiling establishes whether shared memory or a binary RPC layer is required.

4. Repository Layout

```
uce/
  apps/desktop/          # Qt/QML desktop shell
  core/                  # project controller, IDs, errors, config
  world/                 # World Graph schemas/adapters
  identity/              # UEI services and evidence fusion
  operations/            # operation model, DAG, state machine
  media/                 # FFmpeg/media adapters
  storage/               # CAS, project snapshots, asset resolver
  gpu/                   # frame surfaces and GPU resource abstractions
  workers/
    media/
    vision/
    inpaint/
    quality/
    export/
  privacy/               # local/cloud policy enforcement
  security/              # package verification, permissions, audit
  models/                # model manifests, adapters, registry
  benchmarks/            # UCE-Bench datasets + runners
  tests/
    unit/
    integration/
    privacy/
    security/
    recovery/
  docs/adr/
  tools/
```

5. Authoritative Project Model

V0 SHALL not store authoritative state inside generated media files, model checkpoints, or worker-local caches. The desktop project controller is the only authority allowed to commit a new project revision.

```
ProjectRevision {
  project_id: UUID
  revision_id: UUID
  parent_revision_id: UUID?
  created_at: timestamp
  usd_root_ref: AssetRef
  timeline_ref: AssetRef
  semantic_index_ref: AssetRef
  operation_log_ref: AssetRef
  policy_ref: AssetRef
  audit_head: hash
}
```

6. Content-Addressable Storage (CAS)

ID	Priority	Design requirement
CAS-001	MUST	Every immutable source or derived binary artifact has a cryptographic content hash.
CAS-002	MUST	Project revisions reference artifacts by logical AssetRef, never by ad-hoc absolute paths.
CAS-003	MUST	CAS writes are atomic: write temp -> hash/verify -> fsync where applicable -> publish.
CAS-004	MUST	Garbage collection removes only blobs unreachable from retained revisions/pins.
CAS-005	SHOULD	Operation cache keys include operation type/version, input hashes, normalized parameters, model hash, and deterministic settings.
CAS-006	MUST	Cache eviction must never delete a source artifact or an artifact reachable from a committed project revision.

```
AssetRef {
  algo: 'sha256' | 'blake3'
  digest: string
  media_type: string
  byte_size: uint64
  logical_role: source | proxy | mask | depth | render | metadata | model_output
}
```

7. Transactional Project Saving and Recovery

1. Build new immutable artifacts in CAS.
2. Validate hashes and schema versions.
3. Create candidate ProjectRevision.
4. Write revision record atomically.
5. Update project HEAD pointer atomically.
6. Only after HEAD commit may old unreferenced cache become GC-eligible.

- Unexpected termination before step 5 leaves the previous revision authoritative.
- Worker outputs not referenced by a committed revision are temporary/orphan artifacts and may be reclaimed.
- Recovery on startup validates HEAD and falls back to the previous valid revision if corruption is detected.

8. Universal Entity Identity (UEI) V0 Schema

```
Entity {
  entity_id: UUID
  semantic_class: string
  user_label: string?
  created_from: ObservationRef
  state: active | hidden | deleted | uncertain
  confidence: float [0,1]
  evidence: [EvidenceRef]
  representations: [RepresentationRef]
  relationships: [RelationshipRef]
  provenance: [ProvenanceRef]
}

EvidenceRef examples:
  segmentation_mask(frame range)
  bounding_box(frame)
  tracked_points(frame range)
  appearance_embedding(versioned)
```

Rule: model track IDs are evidence metadata only; entity_id never changes because a model backend changes.

9. World/Scene State in V0

V0 does not require full reconstructed 3D world state. It SHALL nevertheless use a schema compatible with later OpenUSD extension. The initial scene stores media source, shots, selected entities, masks/tracks, operation relationships, and timing.

ID	Priority	Design requirement
WORLD-001	MUST	Every operation target references an Entity ID or explicit media range, not a model-private index.
WORLD-002	MUST	Relationships are typed and directional.
WORLD-003	MUST	Observed, estimated, and

		generated representations are distinguishable.
WORLD-004	SHOULD	OpenUSD applied schemas are the target interoperability path; V0 may use a lightweight adapter until the schema plugin is stable.

10. Operation Model and State Machine

```
Operation states:
CREATED -> VALIDATED -> QUEUED -> RUNNING -> VERIFYING ->
  COMMITTED
  FAILED
  CANCELLED
  NEEDS_REPAIR -> REPAIRING -> VERIFYING

Operation {
  operation_id: UUID
  type: string
  schema_version: int
  target_refs: [...]
  input_assets: [...]
  parameters: canonical JSON
  policy_context: PolicySnapshot
  implementation: CapabilitySelection
  status: OperationStatus
  output_assets: [...]
  quality_result: QualityResult?
  provenance: ProvenanceRecord
}
```

- Operation parameters SHALL be canonicalized before cache hashing.
- Cancellation SHALL be cooperative and leave the previous committed project state untouched.
- Failed operations SHALL preserve diagnostics and temporary artifacts according to debug policy without committing semantic state.

11. V0 Capability Contracts

Capability	Input	Output	Notes
media.probe	AssetRef(source)	MediaDescriptor	No decode required
media.proxy	AssetRef(source), ProxySpec	AssetRef(proxy)	Deterministic
vision.segment_frame	frame/time, point/box prompt	MaskArtifact + confidence	Creates selection evidence
vision.propagate_mask	source, seed mask, range	TemporalMaskArtifact + confidence	Backend swappable
edit.remove_entity	source, temporal mask, range	Candidate video frames + reconstruction metadata	Reconstruction-first
quality.preservation	source, candidate, edit	EIS components + defect	No project commit

	region	map	
quality.temporal	candidate, region/range	temporal score + defect map	No project commit
repair.localized	candidate, defect map	repaired candidate	Only affected region/range
media.export	project revision, ExportSpec	final media AssetRef	Preserve audio unless modified

12. Reconstruction-First Object Removal Pipeline

```
Select entity
 -> propagate temporal mask
 -> mask cleanup / dilation policy
 -> identify recoverable pixels from adjacent frames / motion
 -> propagate/reconstruct visible background evidence
 -> generate/inpaint only unresolved regions
 -> composite temporally
 -> quality evaluation
 -> localized repair if needed
 -> commit result as operation output
```

V0 shall not claim physically complete interaction deletion. Interaction-aware restoration (deformations, reflections, linked audio, etc.) is a later dependency-graph milestone, but V0 interfaces must not preclude it.

13. Edit Locks in V0

ID	Priority	Design requirement
LOCK-001	MUST	Preservation region outside the edit mask is implicitly locked for V0 remove operations.
LOCK-002	MUST	Quality evaluation measures unintended change outside the edit region.
LOCK-003	SHOULD	Users may optionally expand protected regions manually.
LOCK-004	FUTURE	Identity, camera, geometry, lighting, motion, and audio semantic locks become explicit in later phases.

14. Quality and Repair Contract

```
QualityResult {
  overall_status: pass | warn | fail
  metrics: {
    edit_isolation: float
    temporal_consistency: float
    reconstruction_confidence: float?
  }
```

```
  defects: [{
    frame_start, frame_end,
    bbox_or_mask_ref,
    type,
    severity,
    confidence,
    suggested_repair
  }]
}
```

- Quality thresholds are benchmark-configured, not hard-coded in UI.
- Localized repair must create a child operation linked to the parent candidate.
- Original successful regions are reused; the repair must not silently replace the entire shot unless explicitly escalated and logged.

15. Privacy Policy Enforcement

```
ExecutionPolicy {
  mode: PRIVATE_LOCAL | PRIVATE_CLOUD | TEAM | PUBLIC
  network_allowed: bool
  third_party_api_allowed: bool
  telemetry_level: NONE | METRICS_ONLY | DIAGNOSTIC
  allowed_endpoints: [...]
}
```

ID	Priority	Design requirement
POL-001	MUST	PRIVATE_LOCAL sets network_allowed=false and blocks external model adapters.
POL-002	MUST	Policy is evaluated before capability routing and before each network-capable worker action.
POL-003	MUST	Model/plugin manifests declare network requirements; undeclared network use is denied.
POL-004	MUST	Privacy mode is captured in Operation provenance.
POL-005	MUST	V0 CI includes a packet-level/offline test proving no unauthorized outbound traffic.

16. Security Boundaries

- Main UI process: trusted, no arbitrary model/plugin execution.
- Workers: isolated process identity, project-scoped capabilities, constrained filesystem roots.
- Model artifacts: hash verified; unsafe Python/pickle loading prohibited in production mode unless explicitly sandboxed.
- IPC messages: schema-validated, size-limited, versioned, and reject unknown privileged fields.
- Archive/project import: path traversal and symlink escape protection mandatory.

- Logs: redact secrets, local private paths where unnecessary, and media content.

17. GPU and Frame Surface Design

```
FrameSurface {
  id: UUID
  width, height
  pixel_format
  color_metadata
  timestamp
  memory_domain: CPU | CUDA | VULKAN | OTHER
  device_id
  handle_or_buffer_ref
  synchronization_token
  ownership: borrowed | shared | owned
}
```

V0 may begin with CPU/OpenCV-compatible surfaces for correctness, but interfaces SHALL avoid assuming CPU-backed frames. A later GPU-native path must not require rewriting operation semantics.

18. Model Manifest and Router

```
ModelManifest {
  id, version, publisher
  capabilities[]
  runtime
  weight_format
  hashes[]
  license_spdx_or_custom
  min_vram
  preferred_vram
  network_required
  filesystem_scope
  benchmark_refs[]
}
```

- Router selection inputs: required capability, privacy policy, hardware, license eligibility, quality baseline, latency, and cost.
- V0 may use a single configured backend per capability but must go through the router interface.
- Model backend names shall not appear in persisted semantic operation types.

19. Logging, Audit, and Provenance

ID	Priority	Design requirement
LOG-001	MUST	Structured logs include operation ID, worker, severity, and trace ID.
LOG-002	MUST	Audit records capture privileged execution, model version/hash, policy decision, input/output

		hashes, and commit result.
LOG-003	MUST	Logs must not contain raw frames, transcripts, credentials, or unrestricted prompts by default.
LOG-004	SHOULD	A single operation trace can be exported for debugging without exporting unrelated project data.

20. Error Model

Class	Examples	Behavior
User/Input	Unsupported codec, invalid selection	Actionable UI error; project remains valid
Resource	OOM, disk full, GPU unavailable	Fail operation; preserve committed state; offer fallback if policy permits
Worker	Crash, timeout, corrupted response	Restart worker if safe; operation fails or resumes from checkpoint
Integrity	Hash mismatch, invalid project revision	Do not commit; quarantine artifact; surface security/integrity warning
Policy	Network denied, license ineligible	Reject before execution
Quality	EIS/temporal failure	Route to repair or require user review

21. V0 User Workflow

1. New Project
2. Import video
3. UCE hashes source and creates CAS entry
4. Probe + proxy generation
5. User seeks to frame and clicks/boxes target
6. Vision worker returns initial mask
7. Project creates UEI entity
8. User confirms target and frame range
9. Vision worker propagates temporal mask
10. Remove operation creates candidate
11. Quality worker evaluates preservation + temporal consistency
12. Repair operation runs if needed/available
13. User compares before/after
14. Commit operation result to project revision
15. Export final media

22. Testing Plan

Suite	V0 target
Unit	CAS hashing, canonical params, UEI IDs, operation state machine, policy evaluation, manifests

Integration	End-to-end local remove workflow
Privacy	Network blocked/private-local assertions
Security	Malformed IPC, path traversal, malicious manifest, unsafe model format rejection
Recovery	Kill workers/UI at every commit stage; disk full; corrupt temp artifact
GPU/resource	OOM handling and worker restart
Benchmark	Static/moving camera, occlusion, low light, fast motion, textured background
Regression	Baseline EIS + temporal scores and failure corpus

23. Initial Benchmark Corpus

- 10 static-camera/simple-background clips.
- 10 moving-camera clips.
- 10 partial-occlusion clips.
- 10 fast-motion clips.
- 10 low-light/noisy clips.
- 10 high-texture/repeating-pattern backgrounds.
- Optional challenge set: shadows/reflections/interactions, marked non-gating for V0.

24. V0 Acceptance Criteria

ID	Priority	Design requirement
AC-001	MUST	Full import-to-export workflow succeeds with networking disabled.
AC-002	MUST	Worker crash does not crash desktop or corrupt last committed revision.
AC-003	MUST	Restart after forced termination restores the last valid revision.
AC-004	MUST	All distributed models/dependencies have recorded licenses and hashes.
AC-005	MUST	No operation directly mutates source media.
AC-006	MUST	UEI remains stable across segmentation and removal operations.
AC-007	MUST	Benchmark gating thresholds are documented before release candidate evaluation.
AC-008	MUST	Security/privacy CI suites pass.

25. Implementation Order

Order	Milestone	Exit condition
1	Repo + CI + coding standards	Build/test pipeline green
2	CAS + transactional project store	Crash-safe create/save/reopen
3	Media probe/proxy/playback	Reliable import and preview
4	Worker framework + IPC	Crash-isolated worker round-trip
5	UEI + selection	Click/box -> stable entity + mask
6	Temporal propagation	Entity mask across shot
7	Removal candidate pipeline	Produces editable candidate
8	Quality/EIS + defect map	Automated verification result
9	Localized repair	At least one automated repair path
10	Export + audit/provenance	Final playable output + trace
11	V0 benchmark gate	Release criteria passed

26. Open Questions Requiring ADR or Spike

Topic	Decision/spike needed before
Primary native language split	Before performance-sensitive media/gpu core
Qt/QML vs alternate desktop shell	Before desktop implementation begins
OpenUSD schema plugin in V0 vs adapter-first	Before semantic project serialization freezes
Hash algorithm SHA-256 vs BLAKE3	Before CAS format becomes persistent
Worker IPC: JSON-RPC vs gRPC/CapnProto/shared memory	Before high-throughput frame transport
Initial segmentation/propagation model	Before vision integration
Initial inpainting backend	Before removal implementation
GPU-native decode path	Before performance milestone; not before correctness prototype

27. Definition of Done for Any V0 Feature

- Requirement/issue linked to implementation.
- Unit/integration tests added.
- Privacy/security impact assessed.
- Failure behavior defined.
- Logs and metrics defined.
- No undocumented persistent format change.
- Benchmark impact measured if quality-affecting.
- License/provenance recorded for new third-party model or code.
- Documentation/ADR updated where architecture changed.